

Introduction aux Design Patterns (Suite)

Rezak AZIZ
rezak.aziz@lecnam.net

CNAM Paris

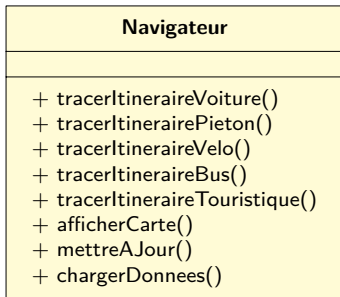
Les patterns de Comportement

Section 1

Strategy

Problème

Une seule classe **Navigateur** contient plusieurs variantes d'un même algorithme : itinéraire voiture, piéton, vélo, bus, etc. Cette classe devient énorme, difficile à maintenir et très fragile.



Problèmes :

- Classe gigantesque
- Multiplication des algorithmes
- Couplage fort et duplication
- Code fragile au moindre changement
- Conflits en équipe (Git)

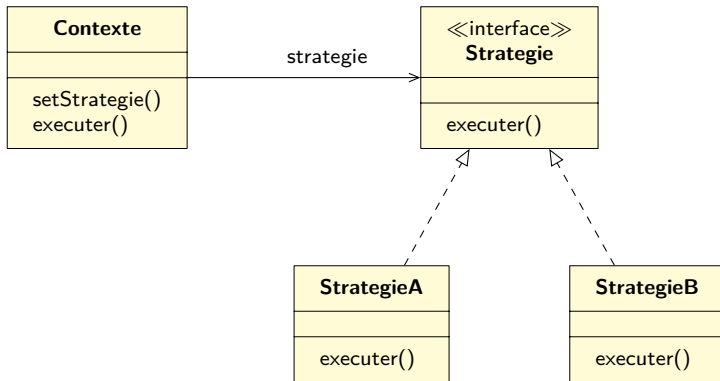
Intention générale

Permettre de définir une **famille d'algorithmes interchangeable**, les encapsuler dans des classes indépendantes et permettre au client de **changer dynamiquement** l'algorithme utilisé par le contexte.

Ce que l'on veut éviter

Les longues structures conditionnelles : `if (mode == voiture) ... else if (mode == velo) ...`

STRATÉGIE — Structure



Quand l'utiliser ?

- Plusieurs variantes d'un même algorithme (tri, compression, IA...).
- Comportement modifiable dynamiquement (jeu, UI, math).
- Remplacer un gros bloc de conditions par du polymorphisme.
- Isolation des algorithmes complexes.
- Respecter le principe *Ouvrir / Fermer*.

STRATÉGIE — Avantages et inconvénients

Avantages

- + Algorithme interchangeable à l'exécution.
- + Réduction des conditions complexes.
- + Facile d'ajouter de nouveaux comportements.
- + Tests unitaires simplifiés.
- + Fortement découplé.

Inconvénients

- - Prolifération de classes.
- - Le client doit connaître la stratégie appropriée.
- - Peut sembler lourd pour de petits algorithmes.
- - Peut être remplacé par des lambdas dans certains langages modernes.

Section 2

Template Method

Problème à résoudre

Observation

Dans de nombreuses applications, plusieurs classes implémentent un algorithme **presque identique**, avec seulement quelques différences.

Exemple : Data Mining (PDF, DOC, CSV)

DocMiner

- + openDoc()
- + extractData()
- + analyzeData()
- + generateReport()

PdfMiner

- + openPdf()
- + extractData()
- + analyzeData()
- + generateReport()

CsvMiner

- + openCsv()
- + extractData()
- + analyzeData()
- + generateReport()

Problèmes

Algorithmes presque identiques → **forte duplication de code**.

Intention du patron

Objectif principal

Définir le **squelette d'un algorithme** dans une classe mère, tout en laissant aux sous-classes la possibilité de **redéfinir certaines étapes**.

Idée clé : On *fixe l'enchaînement* des opérations, mais on laisse varier *leur contenu*.

Ce que l'on évite

```
if (format == PDF) ... else if (format == CSV) ...
```

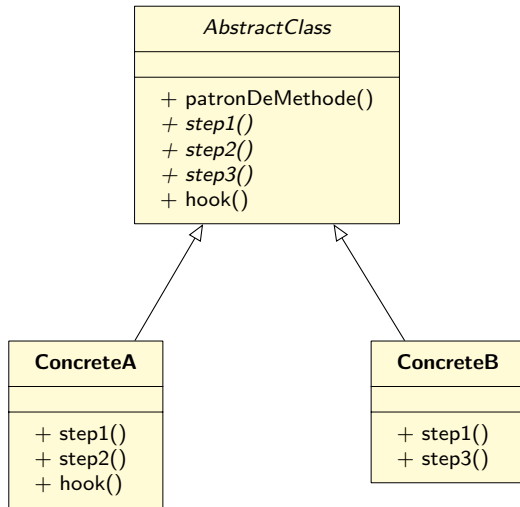
Principe

- Décomposer l'algorithme en étapes.
- Placer l'enchaînement de ces étapes dans une méthode **finale** : la méthode patron.
- Déclarer certaines étapes comme :
 - ▶ abstraites → à implémenter par les sous-classes ;
 - ▶ facultatives → avec une implémentation par défaut ;
 - ▶ crochets (hooks) → implémentation vide pouvant être redéfinie.

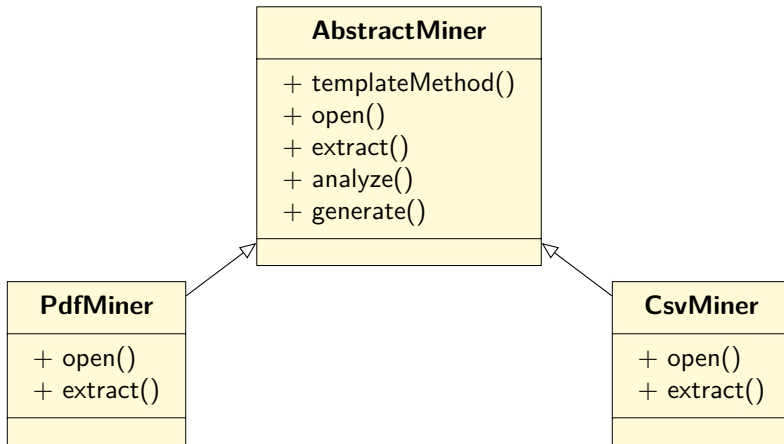
Conséquence

La structure de l'algorithme est **fixée**, mais ses étapes peuvent varier selon les sous-classes.

PATRON DE MÉTHODE — Structure



Structure du Template Method



Quand utiliser ce patron ?

- Lorsqu'un algorithme est identique dans plusieurs classes, avec seulement quelques variations.
- Lorsque vous voulez :
 - ▶ **partager du code** entre les sous-classes ;
 - ▶ **forcer un ordre d'exécution** ;
 - ▶ fournir des **points d'extension contrôlés**.
- Lorsqu'un client veut étendre seulement certaines étapes.
- Lorsque vous voulez remplacer un gros algorithme monolithique par des étapes bien définies.

Avantages

+ Code factorisé

Les étapes communes sont dans la classe mère.

+ Structure stable

L'algorithme est clair et figé.

+ Extensibilité contrôlée

Les sous-classes personnalisent seulement ce qui doit varier.

+ Polymorphisme

Le client n'a plus besoin de if/else ou switch.

Inconvénients

- Structure rigide

Impossible de modifier l'ordre des étapes.

- Risque de LSP violation

Si une étape “par défaut” n'a pas de sens pour une sous-classe.

- Peut devenir complexe

Beaucoup d'étapes → classe abstraite difficile à maintenir.

Section 3

Visiteur

Une application de données géographiques.

Situation

On développe une application orientée objet qui manipule une **structure complexe d'objets**.

- Les objets sont organisés en :
 - ▶ arbre
 - ▶ graphe
 - ▶ structure composite
- Chaque objet représente une entité différente



- Les objets peuvent être :
 - ▶ une **Ville**
 - ▶ une **Usine**
 - ▶ un **Site touristique**
- Tous ces objets sont reliés entre eux

Problème — Nouveau besoin

Nouvelle demande

Implémenter l'export du graphe au format **XML**.

- Le graphe est composé de **plusieurs types d'objets** (ville, usine, site touristique, etc.).
- Chaque type d'objet doit être **exporté différemment**.



Quels solutions proposez vous pour résoudre ce problème ?

Solution naïve n°1

Idée

Ajouter une méthode `exportXML()` dans chaque classe.

- `City.exportXML()`
- `Industry.exportXML()`
- `SightSeeing.exportXML()`

Problèmes

- Les classes de nœud sont déjà en **production**
- L'architecte interdit toute modification
 - ▶ Risque élevé d'introduire des **bugs**
 - ▶ Le code d'export XML rajoute une responsabilité aux classes (Violation SRP).
 - ▶ Chaque nouveau format (JSON, CSV,...) impliquerait de **modifier toutes les classes**

Solution naïve n°2

Idée

Créer une **classe externe** dédiée à l'export XML. Parcourir le graphe depuis cette classe.
Traiter chaque type de nœud séparément

Approche conditionnelle

```
if (obj instanceof City) { ... }  
else if (obj instanceof Industry) { ... }
```

Problèmes

- La classe d'export doit connaître **tous les types de nœuds**
- Utilisation de if/else ou switch sur les types
- Code fortement **couplé aux classes concrètes**
- Violation du principe **Open/Closed**

Contraintes du problème

Ce que nous voulons

- Ajouter de nouveaux comportements
- Éviter les `if/else`
- Respecter le polymorphisme

Ce que nous refusons

- Modifier les classes existantes
- Coupler le code aux types concrets
- Multiplier les responsabilités

Intention du patron Visiteur

Objectif

Séparer les **algorithmes** des **objets** sur lesquels ils opèrent.

Principe

- Classes de nœuds **inchangées**
- Traitements déplacés dans des **visiteurs**
- Ajout d'opérations sans modifier les objets

Problème résolu

- Code déjà en production
- Modifications interdites
- Éviter les if/else

Exemples d'utilisation

- Export XML
- Export JSON
- Statistiques

Comment le Visiteur résout le problème

Idée clé

Le patron Visiteur repose sur une technique appelée **double dispatch**, qui permet d'appeler la bonne méthode **sans utiliser de blocs conditionnels**.

Principe

- Le client ne choisit plus la méthode à appeler
- La décision est déléguée aux objets eux-mêmes
- Chaque nœud connaît sa propre classe
- Il choisit donc la méthode adaptée du visiteur

Mécanisme : `accept(visitor)`

- Le nœud **accepte** un visiteur
- Il appelle la méthode du visiteur correspondant à son type

Single dispatch vs Double dispatch

Single dispatch (cas classique)

- Mécanisme par défaut en POO
- La méthode appelée dépend du type de l'objet receveur

`obj.method(param)`

- La méthode choisie dépend de `obj`

Double dispatch (Visiteur)

- La méthode appelée dépend :
 - ▶ du type du nœud
 - ▶ du type du visiteur
- Deux appels successifs

`node.accept(visitor)`

- 1er dispatch : sur le type du nœud
- 2e dispatch : sur le type du visiteur

Message clé

Le double dispatch permet de choisir une méthode en fonction de **deux types dynamiques**

Double dispatch : exemple de code

Code client

```
foreach (Node node in graph)
    node.accept(exportVisitor)
```

Implémentation côté nœuds

```
// City
class City is
    method accept(Visitor v) is
        v.doForCity(this)

// Industry
class Industry is
    method accept(Visitor v) is
        v.doForIndustry(this)
```

Remarque importante

Oui, les classes nœud sont légèrement modifiées, mais cette modification est :

- minimale
- stable
- et permet d'ajouter de nouveaux comportements **sans toucher aux nœuds**

Conséquence clé du patron Visiteur

Généralisation

- Tous les visiteurs implémentent une **interface Visitor**
- Les nœuds acceptent **n'importe quel visiteur**

Ajouter un nouveau comportement

- Créer une nouvelle classe visiteur
- Implémenter les méthodes pour chaque type de nœud
- Aucun changement dans les classes existantes

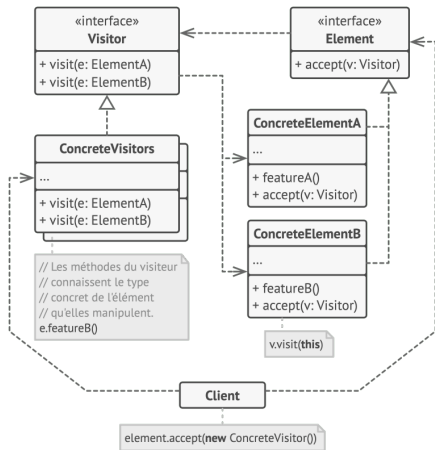
Message clé

Structures stables, comportements extensibles

Structure du patron Visiteur

Éléments principaux

- **Visitor**
 - ▶ Interface des visiteurs
 - ▶ Une méthode par type de nœud
- **ConcreteVisitor**
 - ▶ Implémente les traitements
 - ▶ (Export XML, JSON, statistiques, etc.)
- **Element**
 - ▶ Déclare la méthode `accept()`
- **ConcreteElement**
 - ▶ Implémente `accept(visitor)`



Fonctionnement

Possibilités d'application du patron Visiteur

Quand utiliser le Visiteur

- La structure d'objets est **stable**
- Les comportements évoluent souvent
- Les classes ne doivent pas être modifiées
- Plusieurs traitements doivent s'appliquer aux mêmes objets

Contextes typiques

- Graphes, arbres, AST
- Structures composites
- Modèles métiers complexes

Exemples concrets

- Export de données (XML, JSON, CSV)
- Validation de structures
- Calcul de statistiques
- Génération de rapports

Attention

- Ajouter un nouveau type de nœud est coûteux
- Tous les visiteurs doivent être mis à jour

Avantages et inconvénients du patron Visiteur

Avantages

- Ajout de nouveaux comportements **sans modifier** les classes existantes
- Respect du principe **Open/Closed**
- Suppression des if/else et switch
- Regroupement du code métier dans des classes dédiées
- Facilite l'ajout de traitements complexes

Inconvénients

- Difficile d'ajouter un **nouveau type de nœud**
- Tous les visiteurs doivent être modifiés
- Couplage fort entre visiteurs et classes visitées
- Ajoute de la complexité au design
- Peu intuitif pour les débutants

À retenir

Le Visiteur est efficace si la structure est **stable** et les comportements **évolutifs**

Merci !